

PHASE 7: AUTHENTICATION AND SECURITY IMPLEMENTATION

Project Title: Academic Credential Verification and Alumni Tracking System

Student: Dusabe Marie Rose

Phase Duration: 14 March – 20 March 2026

1. Introduction

Phase 7 focused on improving the security of the backend application by implementing authentication and authorization mechanisms. The goal of this phase was to ensure that only authorized users can access certain parts of the system and that sensitive data is protected from unauthorized access.

Security was implemented using Spring Security together with JSON Web Token (JWT). These technologies allow the system to authenticate users during login and generate tokens that are used to verify user identity when accessing protected API endpoints.

Additionally, password encryption was implemented using BCrypt to ensure that user passwords are securely stored in the database. At the end of this phase, the backend application was able to provide a secure login system and protect sensitive API endpoints.

2. Understanding Spring Security Concepts

The first step of this phase involved studying the security features provided by Spring Security. This included understanding authentication mechanisms, authorization rules, and how secure communication between the client and server is implemented.

Authentication refers to verifying the identity of a user attempting to access the system, while authorization determines what actions the authenticated user is allowed to perform. Understanding these concepts was essential before implementing security features in the backend application.

Figure 1: Spring Security Dependency Configuration

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

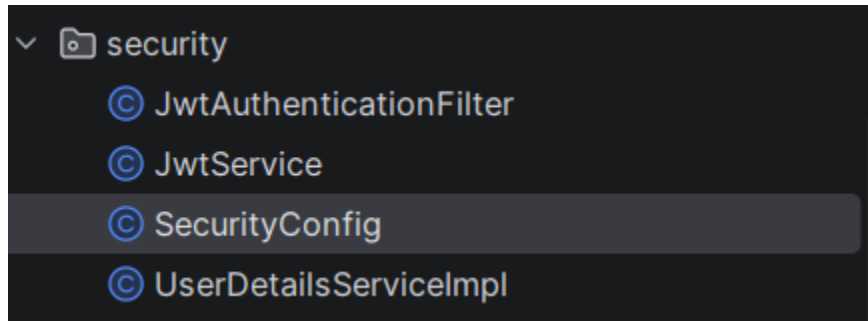
3. Configuring Spring Security in the Backend Project

After understanding the basic security concepts, Spring Security was configured in the backend application. Security configuration classes were created to define authentication rules and protect API endpoints.

The configuration determines which endpoints are publicly accessible and which ones require authentication. For example, login endpoints are accessible to all users, while administrative operations require proper authorization.

This configuration ensures that unauthorized users cannot access protected resources.

Figure 2: Security Configuration Class



4. Implementing User Roles

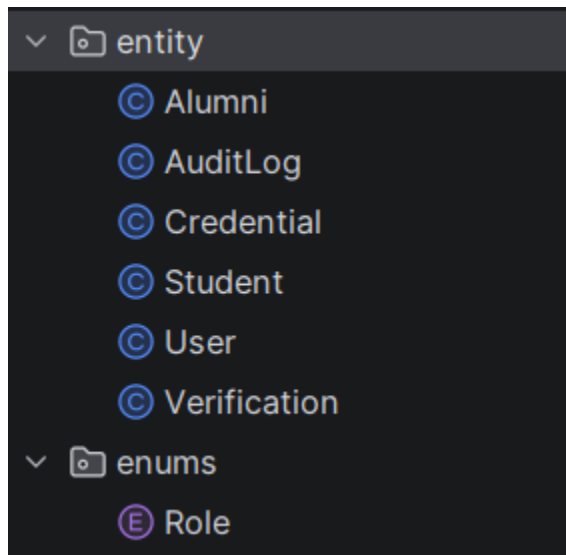
To control access to different parts of the system, role-based authorization was implemented. Three main roles were defined in the system:

- ADMIN
- ALUMNI
- EMPLOYER

Each role has different permissions within the system. Administrators can manage users and system data, alumni users can manage their personal information and credentials, while employers can verify academic credentials through the system.

Role-based access control helps maintain proper system security and ensures that users only access features relevant to their role.

Figure 3: User Entity with Role Attribute



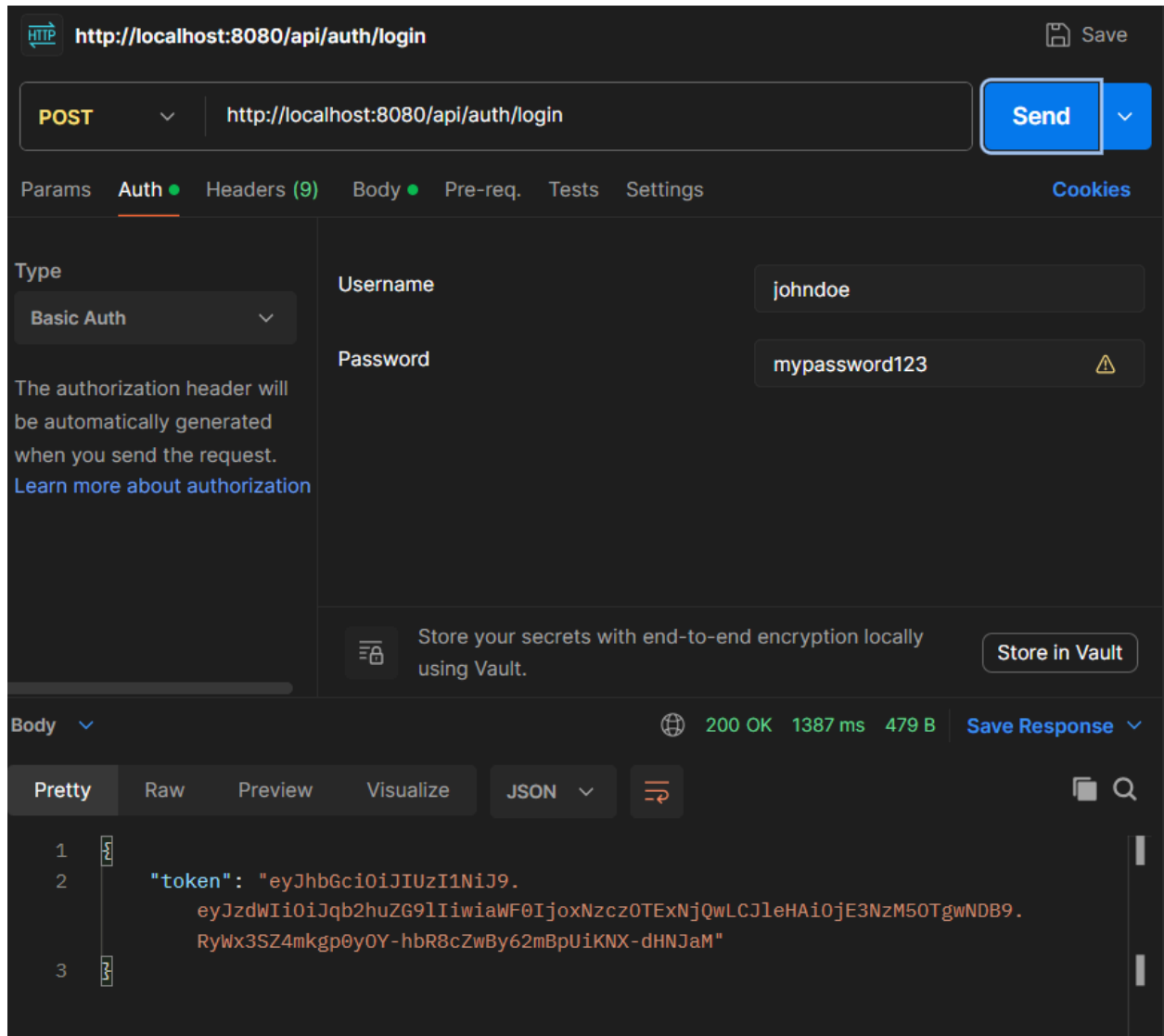
5. Implementing JWT Authentication

JWT authentication was implemented to provide secure communication between the client and the backend server. When a user logs into the system with valid credentials, the backend generates a JWT token.

This token contains encoded user information and is returned to the client application. The client must include this token in the Authorization header when sending requests to protected API endpoints.

The server verifies the token before processing the request. If the token is valid, the request is allowed; otherwise, access is denied.

Figure 4: JWT Token Generated After Login



6. Securing API Endpoints

After implementing authentication, API endpoints were secured using role-based access rules. Specific endpoints were restricted to certain user roles to prevent unauthorized access.

For example, administrative endpoints are accessible only to users with the ADMIN role, while alumni users can only access endpoints related to their personal information.

This approach ensures that the system enforces strict access control policies.

Figure 5: Protected API Endpoint Configuration

```
http
.csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
    .requestMatchers(👁️"/api/auth/**").permitAll()
    .requestMatchers(👁️"/api/admin/**").hasRole("ADMIN")
    .requestMatchers(👁️"/api/alumni/**").hasAnyRole( ...roles: "ADMIN", "ALUMNI")
    .requestMatchers(👁️"/api/employer/**").hasRole("EMPLOYER")
    .requestMatchers(👁️"/api/verification/**").permitAll() // Secure internally via PreAuthoriz
    .anyRequest().authenticated()
)
.sessionManagement( SessionManagementConfigurer<HttpSecurity> session -> session.sessionCreationPolicy(S
.authenticationProvider(authenticationProvider())
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

7. Password Encryption with BCrypt

To improve data security, user passwords were encrypted using BCrypt before being stored in the database. Encryption ensures that even if the database is compromised, the original passwords cannot be easily retrieved.

When a user logs in, the system compares the encrypted password stored in the database with the encrypted version of the password provided during login.

Using BCrypt significantly improves the overall security of the authentication system.

Figure 6: BCrypt Password Encoder Implementation

```
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

8. API Endpoints Implemented

The following endpoints were created during this phase:

1. Register User

POST /api/auth/register

2. Login User

POST /api/auth/login

These endpoints allow users to create accounts and authenticate securely using JWT tokens.

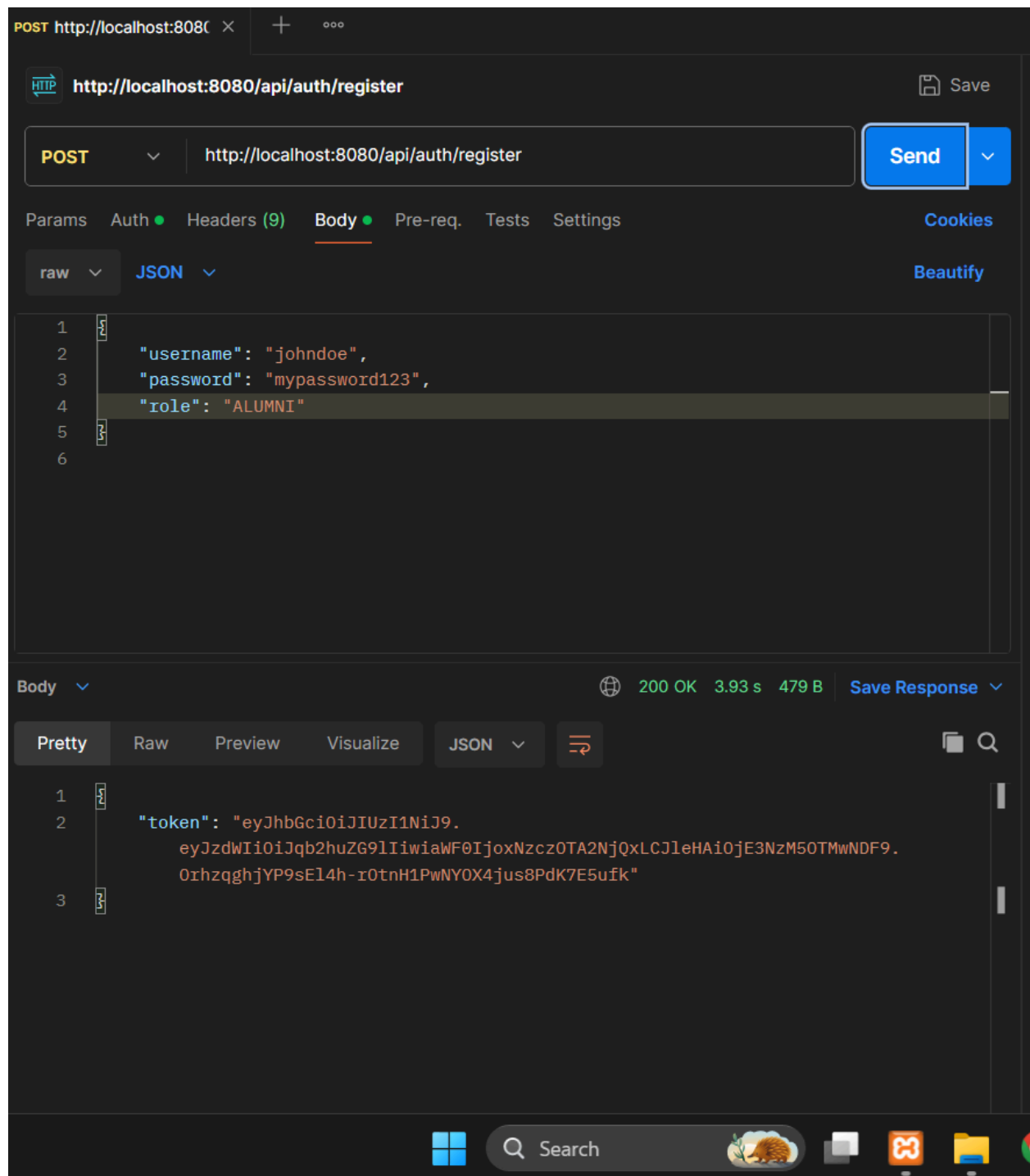
9. Testing Authentication and Authorization

The authentication system was tested using Postman. Several tests were performed to verify that the security configuration works correctly.

- Users can successfully register
- Users can log in and receive JWT tokens
- Tokens are used in Authorization header (Bearer Token)
- Unauthorized users are denied access to protected endpoints

This testing process ensured that authentication and authorization mechanisms function correctly.

Figure 7: Postman Authorization Header with JWT Token

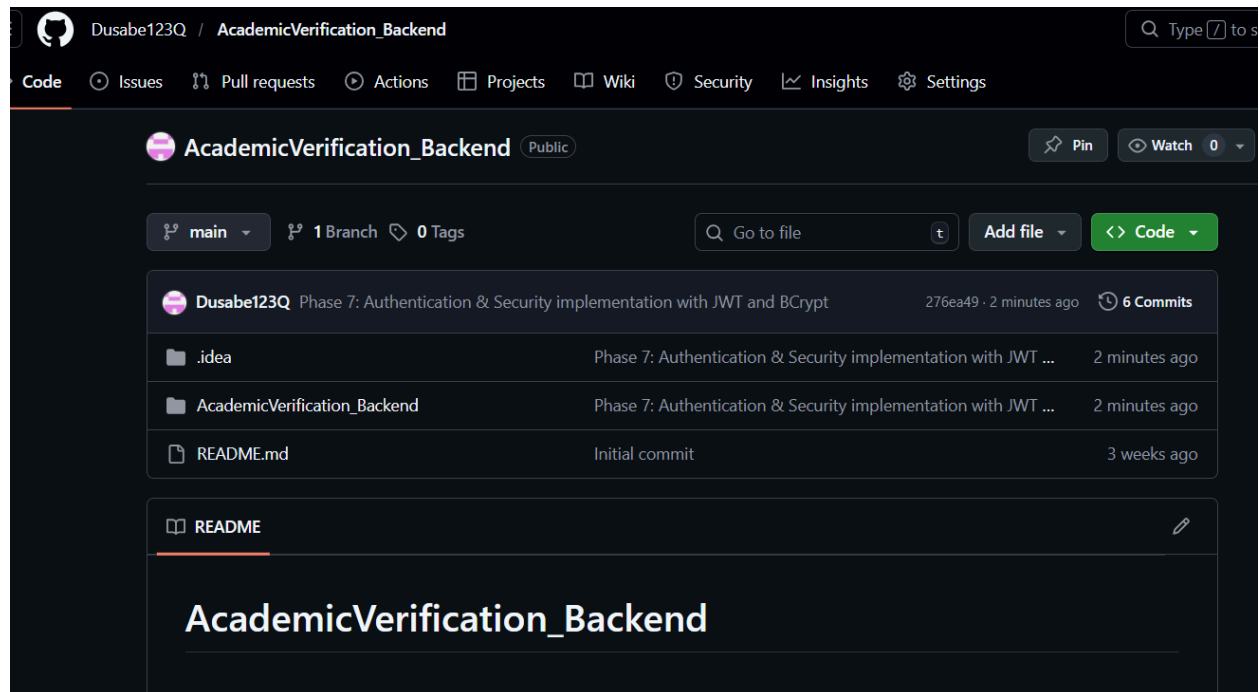


10. Publishing Secure Backend to GitHub

After completing the implementation and testing of authentication and security features, the updated backend project was pushed to the remote repository hosted on GitHub.

This ensures that the latest version of the project containing security improvements is stored and accessible online for future development and review.

Figure 8: GitHub Commit for Phase 7



11. Conclusion

Phase 7 successfully achieved its objective of implementing a secure authentication and authorization system. The backend application now supports:

- Secure user registration and login
- JWT-based authentication
- Role-based access control
- Encrypted password storage using BCrypt

These features ensure that the system is protected against unauthorized access and provide a strong foundation for further development.

